



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет автоматики
и вычислительной техники**

Кафедра вычислительной техники
Лабораторная работа № 3
по дисциплине «Технология разработки программного обеспечения»

**Java. Разработка интерфейса и фабрики для произвольного
типа данных**

Студент Чумаков Илья

Группа АТМ-25

Преподаватель Токарев Вадим Геннадьевич

Новосибирск, 2025

1. Цель работы

Разработать структуру данных (хеш-таблица), универсальную для произвольных типов данных, используя интерфейсы `UserType` и `Comparator`, а также продемонстрировать работу дерева на примере объектов собственного класса `SparseMatrix` и стандартного типа `double`.

2. Разработка интерфейсов

1. `Comparator`

```
public interface Comparator {  
    int compare(Object o1, Object o2);  
}
```

- Позволяет реализовать метод сравнения двух объектов произвольного типа.
- Используется для упорядочивания элементов в хэш-таблице.

2. `UserType`

```
public interface UserType {  
    String typeName(); // Имя типа  
    Object create(); // Создание нового объекта  
    Object clone(Object obj); // Клонирование объекта  
    Object readValue(InputStreamReader in); // Создание и чтение объекта  
    Object parseValue(String ss); // Создание объекта из строки  
    Comparator getTypeComparator(); // Компаратор для сравнения объектов  
}
```

- Обеспечивает универсальный доступ к операциям создания, копирования и сравнения объектов.
- Интерфейс применим как к собственным типам (например, `SparseMatrix`), так и к стандартным (`int`, `double`, `String`).

3. Класс `SparseMatrix`

Класс представляет разреженную матрицу в координатном формате (COO): хранятся только ненулевые элементы в виде списка троек (строка, столбец, значение). Значение элемента имеет тип `double`.

Реализованы основные операции:

- Конструкторы: пустая матрица, матрица заданного размера, матрица из плотного представления `double[][]`.
- `add(SparseMatrix b)` – сложение двух матриц одинакового размера.
- `multiply(SparseMatrix b)` – умножение матриц (размеры согласованы).
- `multiply(double scalar)` – умножение матрицы на скаляр.
- `transpose()` – транспонирование.
- `setElem(int x, int y, double v)` – установка/удаление элемента (если `v = 0`, элемент удаляется).
- Сохранение/загрузка в текстовом и бинарном форматах (`saveToText`, `saveToBin`, `loadFromTxt`, `loadFromBin`).
- `printMatrixDense()` – вывод матрицы в плотном виде для отладки.
- `matricesEqual(SparseMatrix other)` – проверка равенства двух матриц.

4. Интерфейс для базового типа

DoubleUserType реализует интерфейс `UserType` для типа `double`:

- `typeName()` возвращает `"double"`.
- `create()` возвращает `0.0`.
- `clone(Object obj)` возвращает копию переданного значения `Double`.
- `readValue(InputStreamReader in)` читает строку из потока и парсит её в `double`.
- `parseValue(String ss)` преобразует строку в объект `Double` с помощью `Double.parseDouble()`.
- `getTypeComparator()` возвращает компаратор, использующий `Double.compare()` для упорядочивания значений по возрастанию.

SparseMatrixUserType реализует интерфейс `UserType` для собственного типа `SparseMatrix`:

- `typeName()` возвращает `"SparseMatrix"`.
- `create()` создаёт пустую матрицу размером `0×0`.

- `clone(Object obj)` выполняет глубокое копирование матрицы (новый объект с теми же размерами и элементами).
- `readValue(InputStreamReader in)` читает матрицу из текстового потока в формате, совместимом с методом `saveToText()` (первая строка — `n m`, далее строки с `x y v`).
- `parseValue(String ss)` создаёт матрицу из строки вида "`n m x1 y1 v1 x2 y2 v2 ...`".
- `getTypeComparator()` возвращает компаратор, сравнивает матрицы по норме.

5. Универсальная структура данных — `GenericHashTable`

Структура данных из лабораторной работы №2 (хеш-таблица с цепочками) была обобщена и переименована в `GenericHashTable`. Теперь она работает с элементами произвольного типа благодаря использованию интерфейса `UserType`.

Класс содержит:

- Массив корзин (`Node[] heads`).
- Поле `UserType userType` — описание типа хранимых элементов.
- Хранение значений как `Object`.

Основные методы:

- `addToEnd(Object value)` – добавление элемента в конец соответствующей корзины.
- `insert(Object value, int index)` – вставка элемента по логическому индексу (с учётом линейного обхода всех элементов).
- `delete(int index)` – удаление элемента по логическому индексу.
- `get(int index)` – получение элемента по логическому индексу.
- `forEach(Callback cb)` – обход всех элементов с выполнением `callback`.
- `sort()` – создание новой отсортированной хеш-таблицы. Все элементы собираются в список, сортируются с использованием компаратора из

userType.getTypeComparator()), затем добавляются в новую таблицу в отсортированном порядке.

- size() – текущий размер таблицы.

Благодаря передаче объекта UserType при создании таблицы структура стала полностью универсальной и может хранить и упорядочивать объекты любого типа, поддерживающего данный интерфейс.

6. Тестирование

В методе main() реализована тестовая последовательность:

1. Разреженные матрицы (собственный тип SparseMatrix)

```
System.out.println("=== Тестирование HashTable с SparseMatrix ===\n");

UserType matrixType = new SparseMatrixType();

HashTable table = new HashTable(10, matrixType);

SparseMatrix m1 = new SparseMatrix(3, 3);
m1.setElem(0, 0, 1.5);
m1.setElem(1, 1, 2.0);
m1.setElem(2, 2, 3.0);
SparseMatrix m2 = new SparseMatrix(3, 3);
m2.setElem(0, 1, 4.0);
m2.setElem(1, 2, 5.0);

SparseMatrix m3 = new SparseMatrix(2, 2);
m3.setElem(0, 0, 10.0);
SparseMatrix m4 = new SparseMatrix(4, 4);
table.addToEnd(m1);
table.addToEnd(m2);
table.addToEnd(m3);
table.addToEnd(m4);

System.out.println("1. Исходное состояние таблицы (SparseMatrix):");
table.forEach(obj -> ((SparseMatrix) obj).printMatrixDense());

table.insert(matrixType.clone(m1), 1);

System.out.println("\nПосле клонирования и вставки m1 на позицию 1:");
table.forEach(obj -> ((SparseMatrix) obj).printMatrixDense());

System.out.println("\nПолучение по индексу:");
System.out.println("Элемент с индексом 0:");
((SparseMatrix) table.get(0)).printMatrixDense();
System.out.println("Элемент с индексом 3:");
```

```

        ((SparseMatrix) table.get(3)).printMatrixDense();

        System.out.println("\nУдаление элементов:");
        table.delete(2);
        table.delete(0);

        System.out.println("После удаления (осталось " + table.size() + " элемен-
тов:");
        table.forEach(obj -> ((SparseMatrix) obj).printMatrixDense());

```

- Проверка вставки и корректности обхода forEach.

- Клонирование матрицы m1 и вставка клона по индексу 1:

```
table.insert(matrixType.clone(m1), 1);
```

- Демонстрируется работа метода clone() и вставки в произвольную пози-
цию. Получение элементов по индексу:

```
System.out.println(table.get(0));System.out.println(table.get(3));
```

- Проверяется корректность линейного индексирования независимо от
хеш-распределения.

2. Числа типа double (стандартный тип)

```

        UserType doubleType = new DoubleType();

        HashTable table2 = new HashTable(5, doubleType);

        table2.addToEnd(doubleType.parseValue("3.14"));
        table2.addToEnd(doubleType.parseValue("-7.5"));
        table2.addToEnd(doubleType.parseValue("10.0"));
        table2.addToEnd(doubleType.parseValue("3.14"));
        table2.addToEnd(doubleType.parseValue("0.0"));
        table2.addToEnd(doubleType.parseValue("-100.5"));

        System.out.println("До сортировки:");
        printBucketState(table2);
        System.out.println();

        table2.insert(doubleType.parseValue("999.9"), 0);

        System.out.println("После вставки 999.9 в начало:");
        printBucketState(table2);
        System.out.println();

        HashTable sortedDouble = table2.sort();
        System.out.println("После сортировки:");
        printBucketState(sortedDouble);

```

```
table2.delete(0);  
table2.delete(table2.size() - 1);  
  
System.out.println("После удаления двух элементов (осталось " +  
table2.size() + "):");  
table2.forEach(v -> System.out.print(v + " "));  
System.out.println();
```

- Проверка корректной сортировки по возрастанию с учётом дубликатов.
- Демонстрируется работа `parseValue()`, компаратора для `double` и метода `sort()`.
- Удаление двух элементов:
table2.delete(0);
table2.delete(table2.size()-1);